

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ  
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»



Направление подготовки  
09.03.01 Информатика и вычислительная техника  
направленность (профиль)

«Технологии разработки программного обеспечения и обработки больших  
данных»

**Проектная работа «Электронная Библиотека»**

Обучающихся 2 курса  
очной формы обучения  
Фролова Андрея Алексеевича  
Курылева Григория Алексеевича  
Киселева Георгия Петровича

Санкт-Петербург

2026

# Оглавление

|                                                            |    |
|------------------------------------------------------------|----|
| Часть 1. База данных. MongoDB. ....                        | 3  |
| 1.1 Описание предметной области. ....                      | 3  |
| 1.2 Реляционное представление предметной области. ....     | 3  |
| 1.3 ER-диаграмма в нотации UML .....                       | 6  |
| 1.4 Нормализация базы данных. ....                         | 6  |
| 1.5 Переход к MongoDB (основная база данных проекта) ..... | 7  |
| 1.6 Структура базы данных MongoDB.....                     | 8  |
| 1.7 Индексация данных .....                                | 10 |
| Часть 2. CRUD и REST API. Backend. ....                    | 12 |
| 2.1 Описание выполненной части .....                       | 12 |
| 2.2 Используемые технологии .....                          | 12 |
| 2.3 Структура backend-части.....                           | 13 |
| 2.4 Подключение к базе данных .....                        | 13 |
| 2.5 REST API .....                                         | 14 |
| 2.6 Запуск через Docker .....                              | 24 |
| Часть 3. Frontend. ....                                    | 26 |
| 3.1 Описание .....                                         | 26 |
| 3.2 Функциональные возможности .....                       | 26 |
| 3.3 Используемые технологии .....                          | 27 |
| 3.4 Взаимодействие с сервером .....                        | 28 |
| 3.5 Структура проекта .....                                | 28 |
| 3.6 Навигация и интеграция.....                            | 31 |

# **Часть 1. База данных. MongoDB.**

## **1.1 Описание предметной области.**

В качестве проекта нам предложена предметная область - Электронная Библиотека. Электронная библиотека представляет собой информационную систему, предназначенную для хранения и управления данными о книгах. Система позволяет вести каталог литературных произведений, хранить сведения об авторах, жанрах и издательствах, а также выполнять поиск и фильтрацию книг по различным параметрам.

Основным объектом предметной области является книга. Для каждой книги необходимо хранить информацию о названии, годе издания и международный идентификатор ISBN. Кроме того, каждая книга может быть связана с одним или несколькими авторами, относиться к нескольким жанрам и быть выпущенной определённым издательством.

Для хранения информации об авторах система содержит сведения об их имени, возможном псевдониме и контактных данных. Для классификации литературных произведений используются жанры, позволяющие группировать книги по тематике и содержанию. Также система хранит информацию об издательствах, выпустивших книги, что позволяет осуществлять поиск и фильтрацию по издательствам.

## **1.2 Реляционное представление предметной области.**

Для проектирования данных удобно было рассмотреть предметную область сначала в реляционной форме. Это помогает выполнить требования по нормализации и ER-диаграмме, а затем адаптировать модель к MongoDB.

В данном разделе представлена концептуальная SQL-модель предметной области и основные сущности будущей базы данных. На текущем

этапе описаны только структуры основных таблиц и их атрибуты. Информация о связях между сущностями намеренно вынесена в следующий раздел и будет представлена в виде ER-диаграммы.

Для хранения книг используем сущность **Book**:

| Название поля | Тип    | Описание                                       |
|---------------|--------|------------------------------------------------|
| id            | PK     | Ключевое поле                                  |
| title         | string | Название книги                                 |
| year          | int    | Год издания                                    |
| ISBN          | string | Международный стандартный книжный номер (ISBN) |

Для хранения авторов используем сущность **Author**:

| Название поля     | Тип    | Описание                                           |
|-------------------|--------|----------------------------------------------------|
| id                | PK     | Ключевое поле                                      |
| author_name       | string | Полное имя автора                                  |
| author_nickname   | string | Псевдоним автора если тот имеется                  |
| email             | string | контактная почта автора, если тот её указал        |
| social_media_link | string | Возможность указать ссылку на свои социальные сети |

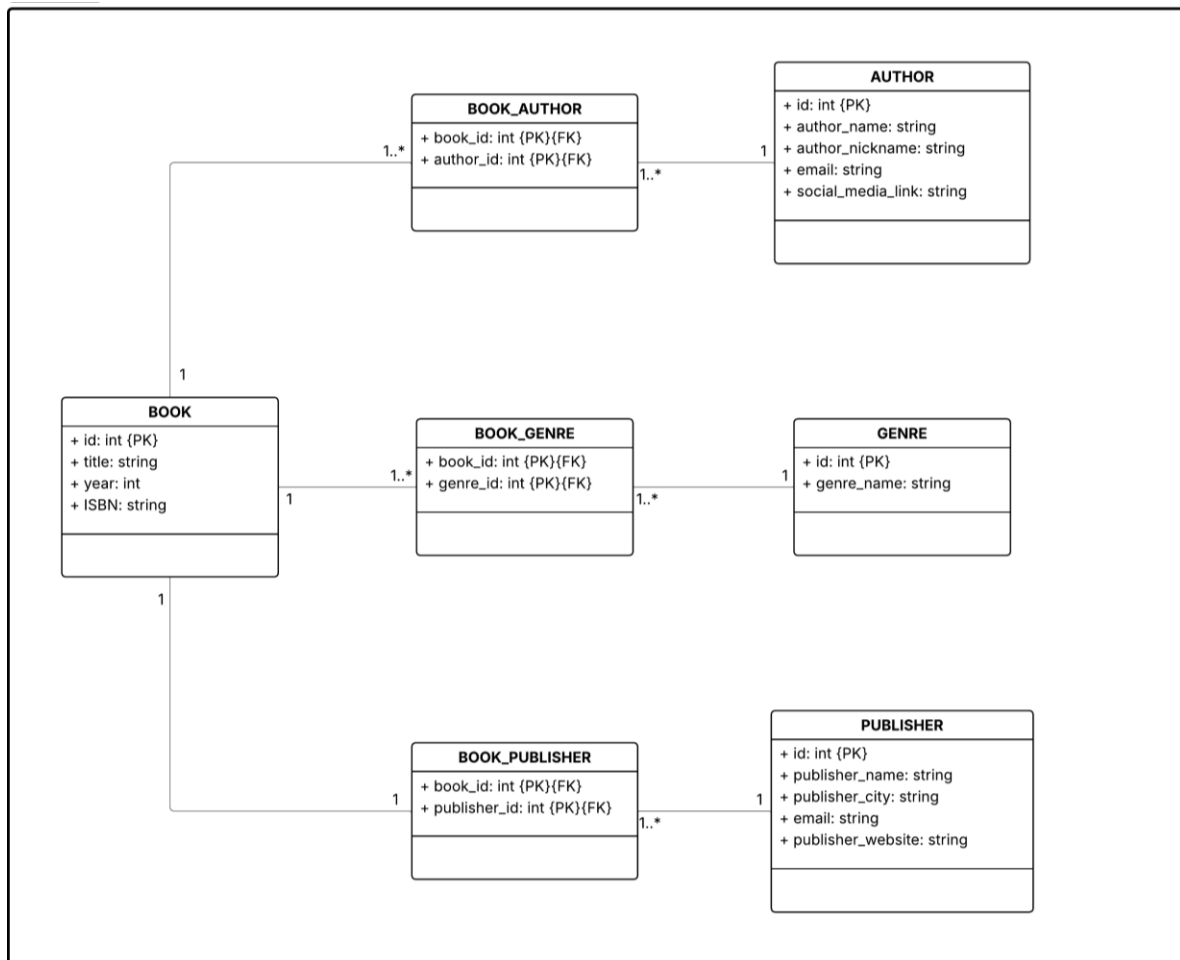
Для хранения информации о жанрах создадим отдельную таблицу **Genre**:

| Название поля | Тип    | Описание       |
|---------------|--------|----------------|
| id            | PK     | Ключевое поле  |
| genre_name    | string | Название жанра |

Для хранения информации об издательствах **Publisher:**

| Название поля     | Тип    | Описание                                          |
|-------------------|--------|---------------------------------------------------|
| id                | PK     | Ключевое поле                                     |
| publisher_name    | string | Название издательства                             |
| publisher_city    | string | Город издательства                                |
| email             | string | контактная почта издательства, если тот её указал |
| publisher_website | string | Вебсайт издательства                              |

## 1.3 ER-диаграмма в нотации UML



### Связи:

- Book M:M Author - у книги может быть несколько авторов, у одного автора может быть много книг
- Book M:M Genre - у книги может быть несколько жанров, один жанр может относиться к нескольким книгам
- Book M:1 Publisher - у книги может быть только одно издательство, у одного издательства может быть много книг

## 1.4 Нормализация базы данных.

Для хранения основных объектов предметной области были выделены отдельные сущности: Book, Author, Genre и Publisher. Такой подход позволяет избежать дублирования информации. Например, данные об авторе или

издательстве хранятся только один раз и могут использоваться для нескольких книг.

Связи типа «многие ко многим» между книгами и авторами, а также между книгами и жанрами реализованы с помощью промежуточных таблиц Book\_Author и Book\_Genre. Это позволяет хранить информацию о нескольких авторах одной книги и нескольких жанрах без дублирования данных.

Каждая таблица содержит атомарные значения атрибутов, что соответствует требованиям первой нормальной формы (1НФ). Все неключевые атрибуты зависят только от первичного ключа, что соответствует второй нормальной форме (2НФ). Также в модели отсутствуют транзитивные зависимости между неключевыми атрибутами, что позволяет считать структуру базы данных, приведённой к третьей нормальной форме (3НФ).

## **1.5 Переход к MongoDB (основная база данных проекта)**

Выбор MongoDB обусловлен тем, что данная СУБД лучше подходит для задач, связанных с хранением слабо структурированных и потенциально расширяемых данных. В рамках системы «Электронная библиотека» предполагается работа с сущностями, которые могут со временем изменяться и дополняться новыми атрибутами (например, рейтингами книг, отзывами пользователей, дополнительными характеристиками авторов или метаданными издательств). В реляционной модели такие изменения требуют модификации схемы таблиц, тогда как в MongoDB структура документа может быть расширена без изменения всей модели данных.

Дополнительным фактором выбора стало то, что основная нагрузка системы приходится на операции чтения: отображение списка книг, фильтрация, поиск по параметрам и просмотр карточки книги. MongoDB обеспечивает высокую эффективность таких операций за счёт хранения

данных в виде документов, максимально приближенных к структуре выдачи пользовательского интерфейса.

## 1.6 Структура базы данных MongoDB

В рамках проекта выделены следующие основные коллекции.

### Books (Книги)

Коллекция является центральной в системе и содержит основную информацию о книгах.

```
{
  "_id": ObjectId(),
  "title": "1984",
  "year": 1949,
  "isbn": "978-5-17-118366-6",
  "author_ids": [
    ObjectId("...")
  ],
  "genre_ids": [
    ObjectId("...")
  ],
  "publisher_id": ObjectId("...")
}
```

### Authors (Авторы)

```
{
  "_id": ObjectId(),
  "author_name": "George Orwell",
  "author_nickname": null,
  "email": "example@mail.com",
  "social_media_link": "https://..."
}
```

### Genres (Жанры)



```
{
  "_id": ObjectId(),
  "genre_name": "Dystopia"
}
```

## **Publishers (Издательства)**

```
{
  "_id": ObjectId(),
  "publisher_name": "AST",
  "publisher_city": "Moscow",
  "email": "contact@publisher.com",
  "publisher_website": "https://..."
}
```

## **Идентификаторы MongoDB**

MongoDB автоматически назначает каждому документу поле `_id`, если вы не указали его вручную. В обычном случае это значение создаётся как `ObjectId` — уникальный 12-байтовый идентификатор.

## **Реализация связей между сущностями**

Связи между сущностями реализованы с использованием идентификаторов документов:

- одна книга может содержать несколько авторов (`author_ids`)
- одна книга может относиться к нескольким жанрам (`genre_ids`)
- каждая книга связана с одним издательством (`publisher_id`)

Такой подход позволяет сохранить логическую целостность данных, при этом избежать избыточности и сложных операций соединения данных, характерных для реляционных СУБД. Дополнительно данный подход обеспечивает баланс между нормализацией и производительностью:

ключевые сущности остаются независимыми, но при этом легко связываются на уровне приложения.

## **1.7 Индексация данных**

Для обеспечения высокой производительности поиска и фильтрации в системе была спроектирована система индексов, ориентированная на типовые пользовательские сценарии работы с электронной библиотекой. Основная нагрузка в приложении приходится на операции чтения данных, включая поиск книги по названию, фильтрацию по году издания, поиск по автору и жанру, просмотр карточки книги и выборку книг конкретного издательства. Так как объём данных в библиотечной системе потенциально может расти, использование индексов является критически важным для поддержания высокой скорости отклика системы.

### **Основные индексы коллекции Books**

- индекс по title — используется для поиска книг по названию
- уникальный индекс по isbn — обеспечивает уникальность и быстрый доступ к книге
- индекс по year — используется для фильтрации по году издания
- индекс по publisher\_id — ускоряет выборку книг издательства
- индекс по author\_ids — ускоряет поиск книг по автору
- индекс по genre\_ids — ускоряет фильтрацию по жанрам

Также для типовых сценариев поиска предусмотрены составные индексы:

- (year, title) — фильтрация по году с последующим поиском по названию
- (publisher\_id, year) — выборка книг издательства за период

### **Индексы в коллекциях Authors и Genres**

- author\_name — ускорение поиска автора по имени
- genre\_name — ускорение фильтрации и поиска по жанрам

- publisher\_name — ускорение поиска издательства по названию

### **Обоснование выбранной стратегии индексации**

При проектировании индексов учитывался баланс между скоростью чтения и стоимостью операций записи.

Так как система ориентирована преимущественно на чтение данных, увеличение количества индексов является оправданным решением. При этом индексы добавлялись только на наиболее часто используемые поля, чтобы избежать избыточной нагрузки на операции вставки и обновления данных.

## Часть 2. CRUD и REST API. Backend.

### 2.1 Описание выполненной части

Серверная часть предоставляет единый программный интерфейс для работы с книгами, авторами, жанрами и издательствами, а также обеспечивает сохранение изменений в базе данных MongoDB.

Основная задача backend-части - связать пользовательский интерфейс и базу данных. Клиентское приложение может отправлять HTTP-запросы на сервер, а сервер выполняет нужные операции с коллекциями MongoDB и возвращает результат в формате JSON.

Реализация находится в папке backend и написана на Python с использованием FastAPI и PyMongo.

### 2.2 Используемые технологии

Для реализации серверной части были использованы следующие технологии:

| Технология | Назначение                                                |
|------------|-----------------------------------------------------------|
| Python     | Основной язык серверной части                             |
| FastAPI    | Создание REST API и автоматической Swagger-документации   |
| PyMongo    | Подключение к MongoDB и выполнение запросов к базе данных |
| Uvicorn    | ASGI-сервер для запуска FastAPI-приложения                |
| Docker     | Контейнеризация                                           |

## 2.3 Структура backend-части

Основные файлы серверной части:

```
backend/  
  app/  
    __init__.py  
    main.py  
  Dockerfile  
  requirements.txt  
  .dockerignore
```

Файл backend/app/main.py содержит основную логику приложения:

- подключение к MongoDB;
- описание FastAPI-приложения;
- настройку CORS;
- маршруты для CRUD-операций;
- фильтрацию, сортировку и пагинацию;
- преобразование ObjectId в строки для JSON-ответов;
- проверку существования связанных документов;
- обработку ошибок.

Файл requirements.txt содержит Python-зависимости проекта, а Dockerfile описывает сборку контейнера backend-сервиса.

## 2.4 Подключение к базе данных

Сервер подключается к MongoDB через переменные окружения:

```
MONGO_URI=mongodb://database:27017  
DB_NAME=electrolibrary
```

В Docker Compose backend обращается к MongoDB по имени сервиса database, поэтому внутри контейнерной сети используется адрес:

```
mongodb://database:27017
```

Если сервер запускается локально без Docker, по умолчанию используется:

```
mongodb://localhost:27017
```

Работа выполняется с базой данных electrolibrary и коллекциями:

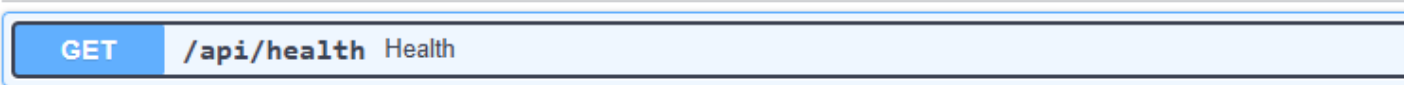
- books;
- authors;
- genres;
- publishers.

## 2.5 REST API

REST API построен вокруг основных сущностей предметной области: книг, авторов, жанров и издательств. Все маршруты начинаются с префикса /api.

Для проверки работоспособности сервера реализован служебный маршрут:

| Метод | Маршрут     | Назначение             |
|-------|-------------|------------------------|
| GET   | /api/health | Проверка состояния API |



GET /api/health Health

Пример ответа:

```
{
  "status": "ok",
  "database": "electrolibrary"
}
```

## CRUD для книг

Книги являются основной сущностью приложения. Для них реализован полный набор CRUD-операций.

| Метод  | Маршрут              | Назначение                            |
|--------|----------------------|---------------------------------------|
| GET    | /api/books           | Получить список книг                  |
| GET    | /api/books/{book_id} | Получить одну книгу по идентификатору |
| POST   | /api/books           | Создать новую книгу                   |
| PATCH  | /api/books/{book_id} | Частично обновить книгу               |
| PUT    | /api/books/{book_id} | Полностью заменить книгу              |
| DELETE | /api/books/{book_id} | Удалить книгу                         |

|        |                      |              |   |
|--------|----------------------|--------------|---|
| GET    | /api/books           | List Books   | ▼ |
| POST   | /api/books           | Create Book  | ▼ |
| GET    | /api/books/{book_id} | Get Book     | ▼ |
| PATCH  | /api/books/{book_id} | Update Book  | ▼ |
| PUT    | /api/books/{book_id} | Replace Book | ▼ |
| DELETE | /api/books/{book_id} | Delete Book  | ▼ |

При получении книги сервер возвращает не только поля самой книги, но и связанные документы:

- авторов из коллекции authors;
- жанры из коллекции genres;

- издательство из коллекции publishers.

Для этого используется агрегация MongoDB с операциями \$lookup и \$unwind.

Пример структуры ответа для книги:

```
{
  "_id": "OBJECT_ID",
  "title": "1984",
  "year": 1949,
  "isbn": "978-0-452-28423-4",
  "author_ids": ["AUTHOR_ID"],
  "genre_ids": ["GENRE_ID"],
  "publisher_id": "PUBLISHER_ID",
  "authors": [
    {
      "_id": "AUTHOR_ID",
      "author_name": "George Orwell"
    }
  ],
  "genres": [
    {
      "_id": "GENRE_ID",
      "genre_name": "Dystopia"
    }
  ],
  "publisher": {
    "_id": "PUBLISHER_ID",
    "publisher_name": "Penguin Books"
  }
}
```



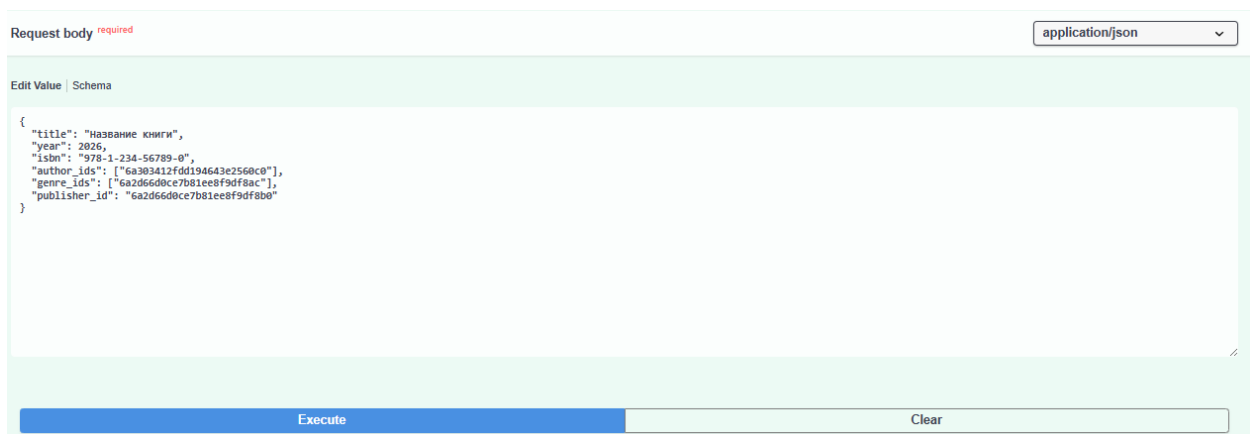


## Создание книги

Для создания книги клиент отправляет POST-запрос на маршрут /api/books.

Пример JSON:

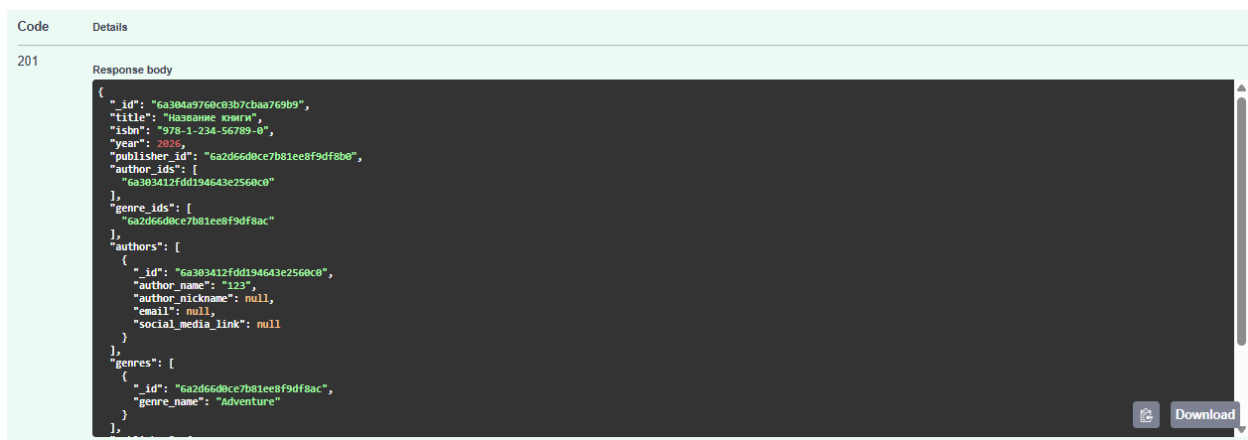
```
{
  "title": "Название книги",
  "year": 2026,
  "isbn": "978-1-234-56789-0",
  "author_ids": ["AUTHOR_ID"],
  "genre_ids": ["GENRE_ID"],
  "publisher_id": "PUBLISHER_ID"
}
```



Перед сохранением сервер выполняет проверки:

- поле title должно быть передано;
- поле year должно быть целым положительным числом;
- поле isbn должно быть передано;
- author\_ids должен быть массивом идентификаторов;
- genre\_ids должен быть массивом идентификаторов;
- publisher\_id должен быть корректным ObjectId;
- связанные авторы, жанры и издательство должны существовать в базе данных.

Если данные корректны, новая книга сохраняется в коллекцию books, а сервер возвращает созданный документ.



The screenshot shows a REST client interface with a status code of 201 and a JSON response body. The response contains the following fields:

```
{
  "_id": "6a304a9760c03b7cbaa769b9",
  "title": "Название книги",
  "isbn": "978-1-234-56789-0",
  "year": 2019,
  "publisher_id": "6a2d66d0ce7b81ee8f9dfab9",
  "author_ids": [
    "6a303412fdd194643e2560c0"
  ],
  "genre_ids": [
    "6a2d66d0ce7b81ee8f9dfaac"
  ],
  "authors": [
    {
      "_id": "6a303412fdd194643e2560c0",
      "author_name": "123",
      "author_nickname": null,
      "email": null,
      "social_media_link": null
    }
  ],
  "genres": [
    {
      "_id": "6a2d66d0ce7b81ee8f9dfaac",
      "genre_name": "Adventure"
    }
  ]
}
```

## Редактирование книги

Для редактирования книги реализованы два метода:

| Метод | Особенность                       |
|-------|-----------------------------------|
| PATCH | Меняет только переданные поля     |
| PUT   | Полностью заменяет документ книги |

Пример частичного обновления:

```
{  
  "title": "Новое название",  
  "year": 2027  
}
```

Если книга с указанным идентификатором не найдена, сервер возвращает ошибку 404.

### Удаление книги

Удаление выполняется запросом:

```
DELETE /api/books/{book_id}
```

Если книга существует, она удаляется из коллекции books, а сервер возвращает статус 204 No Content. Если книги с таким идентификатором нет, возвращается ошибка 404.

### Фильтрация, сортировка и пагинация книг

Для списка книг реализованы параметры запроса, которые позволяют искать, фильтровать и сортировать данные.

| Параметр | Назначение                     |
|----------|--------------------------------|
| search   | Поиск по названию книги        |
| year     | Фильтр по точному году издания |
| yearFrom | Минимальный год издания        |
| yearTo   | Максимальный год издания       |

| Параметр     | Назначение                           |
|--------------|--------------------------------------|
| publisher_id | Фильтр по издательству               |
| author_id    | Фильтр по автору                     |
| genre_id     | Фильтр по жанру                      |
| sort         | Поле сортировки                      |
| order        | Направление сортировки: asc или desc |
| page         | Номер страницы                       |
| limit        | Количество элементов на странице     |

Примеры запросов:

```
GET /api/books?search=1984
GET /api/books?year=1949
GET /api/books?yearFrom=1930&yearTo=1950
GET /api/books?author_id=OBJECT_ID
GET /api/books?genre_id=OBJECT_ID
GET /api/books?publisher_id=OBJECT_ID
GET /api/books?sort=year&order=desc&page=1&limit=10
```

## CRUD для справочников

Помимо книг реализован CRUD для справочных сущностей: авторов, жанров и издательств.

### Авторы

| Метод  | Маршрут                | Назначение                        |
|--------|------------------------|-----------------------------------|
| GET    | /api/authors           | Получить список авторов           |
| GET    | /api/authors/{item_id} | Получить автора по идентификатору |
| POST   | /api/authors           | Создать автора                    |
| PATCH  | /api/authors/{item_id} | Частично обновить автора          |
| PUT    | /api/authors/{item_id} | Полностью заменить автора         |
| DELETE | /api/authors/{item_id} | Удалить автора                    |

|        |                        |                |   |
|--------|------------------------|----------------|---|
| GET    | /api/authors           | List Entities  | ⌵ |
| POST   | /api/authors           | Create Entity  | ⌵ |
| GET    | /api/authors/{item_id} | Get Entity     | ⌵ |
| PATCH  | /api/authors/{item_id} | Update Entity  | ⌵ |
| PUT    | /api/authors/{item_id} | Replace Entity | ⌵ |
| DELETE | /api/authors/{item_id} | Delete Entity  | ⌵ |

Поля автора:

```
{
  "author_name": "Имя автора",
  "author_nickname": "Псевдоним",
  "email": "author@example.com",
  "social_media_link": "https://example.com"
}
```

```
{
  "_id": "6a2d66d0ce7b81ee8f9df8a5",
  "author_name": "J.R.R. Tolkien",
  "author_nickname": "Tolkien",
  "email": "tolkien@mail.com",
  "social_media_link": "https://www.example.com/jrr-tolkien"
}
```

## Жанры

| Метод  | Маршрут               | Назначение                      |
|--------|-----------------------|---------------------------------|
| GET    | /api/genres           | Получить список жанров          |
| GET    | /api/genres/{item_id} | Получить жанр по идентификатору |
| POST   | /api/genres           | Создать жанр                    |
| PATCH  | /api/genres/{item_id} | Частично обновить жанр          |
| PUT    | /api/genres/{item_id} | Полностью заменить жанр         |
| DELETE | /api/genres/{item_id} | Удалить жанр                    |

|        |                       |                |   |
|--------|-----------------------|----------------|---|
| GET    | /api/genres           | List Entities  | ▼ |
| POST   | /api/genres           | Create Entity  | ▼ |
| GET    | /api/genres/{item_id} | Get Entity     | ▼ |
| PATCH  | /api/genres/{item_id} | Update Entity  | ▼ |
| PUT    | /api/genres/{item_id} | Replace Entity | ▼ |
| DELETE | /api/genres/{item_id} | Delete Entity  | ▼ |

Поля жанра:

```
{  
  "genre_name": "Fantasy"  
}
```

```
{  
  "_id": "6a2d66d0ce7b81ee8f9df8ac",  
  "genre_name": "Adventure"  
},  
/
```

## Издательства

| Метод  | Маршрут                   | Назначение                              |
|--------|---------------------------|-----------------------------------------|
| GET    | /api/publishers           | Получить список издательств             |
| GET    | /api/publishers/{item_id} | Получить издательство по идентификатору |
| POST   | /api/publishers           | Создать издательство                    |
| PATCH  | /api/publishers/{item_id} | Частично обновить издательство          |
| PUT    | /api/publishers/{item_id} | Полностью заменить издательство         |
| DELETE | /api/publishers/{item_id} | Удалить издательство                    |

|        |                           |                |   |
|--------|---------------------------|----------------|---|
| GET    | /api/publishers           | List Entities  | ▼ |
| POST   | /api/publishers           | Create Entity  | ▼ |
| GET    | /api/publishers/{item_id} | Get Entity     | ▼ |
| PATCH  | /api/publishers/{item_id} | Update Entity  | ▼ |
| PUT    | /api/publishers/{item_id} | Replace Entity | ▼ |
| DELETE | /api/publishers/{item_id} | Delete Entity  | ▼ |

### Поля издательства:

```
{
  "publisher_name": "Название издательства",
  "publisher_city": "Город",
  "email": "publisher@example.com",
  "publisher_website": "https://example.com"
}
```

```
{
  "_id": "6a2d66d0ce7b81ee8f9df8b0",
  "publisher_name": "Bloomsbury",
  "publisher_city": "London",
  "email": "support@bloomsbury.com",
  "publisher_website": "https://www.bloomsbury.com"
},
{
```

## Поиск и сортировка справочников

Для справочников доступен поиск по строке q.

Примеры:

```
GET /api/authors?q=orwell
GET /api/genres?q=fantasy
GET /api/publishers?q=penguin.
```

## Обработка ошибок

Сервер возвращает понятные HTTP-статусы:

| Статус | Ситуация                                            |
|--------|-----------------------------------------------------|
| 400    | Некорректные входные данные                         |
| 404    | Документ не найден                                  |
| 409    | Нарушение уникальности, например повторяющийся ISBN |
| 500    | Внутренняя ошибка сервера                           |

## 2.6 Запуск через Docker

Backend запускается через Docker Compose вместе с MongoDB. В docker-compose.yml добавлен сервис api, который собирается из папки backend.

Фрагмент конфигурации:



```
api:
  build:
    context: ./backend
  environment:
    PORT: 3000
    MONGO_URI: mongodb://database:27017
    DB_NAME: electrolibrary
  ports:
    - 3000:3000
  depends_on:
    - database
```

Команда запуска проекта:

```
docker compose up --build
```

После запуска API доступен по адресу:

```
http://localhost:3000
```

```
%PS D:\Study\Main Uni\repos\Study-Library-Project> docker compose up
[+] up 4/4
✓ Network study-library-project_default Created 0.0s
✓ Container study-library-project-database-1 Created 0.1s
✓ Container study-library-project-api-1 Created 0.1s
✓ Container study-library-project-frontend-1 Created 0.1s
```

## Часть 3. Frontend.

### 3.1 Описание

Frontend-часть проекта **ElectroLibrary**, реализованная с использованием **Vue 3**, **Vue Router** и **Axios**.

Приложение предоставляет удобный пользовательский интерфейс для взаимодействия с серверной частью электронной библиотеки и позволяет выполнять основные операции управления коллекцией книг.

Дизайн интерфейса выполнен в минималистичном стиле.

### 3.2 Функциональные возможности

#### Работа с книгами

- Просмотр списка книг.
- Просмотр подробной информации о книге.
- Создание новых записей.
- Редактирование существующих книг.
- Удаление книг из коллекции.

#### Управление справочниками

- **Авторы:** добавление, редактирование и удаление авторов библиотеки.
- **Жанры:** управление жанрами для категоризации книг.
- **Издательства:** управление информацией об издательствах.

#### Поиск и фильтрация

- Поиск книг по названию.
- Фильтрация по автору.
- Фильтрация по издательству.
- Фильтрация по жанру.

- Сортировка списка книг по различным параметрам.
- Поиск и сортировка авторов, жанров и издательств.

## Навигация

Для организации многостраничного интерфейса используется Vue Router.

Доступные маршруты:

| Маршрут     | Описание                       |
|-------------|--------------------------------|
| /           | Перенаправление на список книг |
| /books      | Список всех книг               |
| /books/:id  | Подробная информация о книге   |
| /authors    | Управление авторами            |
| /genres     | Управление жанрами             |
| /publishers | Управление издательствами      |

## 3.3 Используемые технологии

### Основные технологии

- Vue 3
- Vue Router
- Axios
- Vite

### Вспомогательные технологии

- HTML5

- CSS3
- JavaScript ES2022

### 3.4 Взаимодействие с сервером

Frontend взаимодействует с REST API серверной части посредством HTTP-запросов.

Основные методы API:

| Метод  | Назначение        |
|--------|-------------------|
| GET    | Получение данных  |
| POST   | Создание записи   |
| PATCH  | Обновление записи |
| DELETE | Удаление записи   |

### 3.5 Структура проекта

```

frontend/
|
├─ public/
|
├─ src/
|   └─ api/
|       └─ libraryApi.js
|       |
|       └─ components/
|           └─ AppButton.vue
|           └─ AuthorForm.vue
|           └─ BaseModal.vue

```

```
| | └─ BookCard.vue
| | └─ BookForm.vue
| | └─ GenreForm.vue
| | └─ EntitySelect.vue
| | └─ PageHeader.vue
| | └─ PublisherForm.vue
| |
| └─ pages/
| | └─ AboutPage.vue
| | └─ AuthorsPage.vue
| | └─ BookDetailsPage.vue
| | └─ BooksPage.vue
| | └─ GenresPage.vue
| | └─ PublishersPage.vue
| |
| └─ router/
| | └─ index.js
| |
| └─ App.vue
| └─ main.js
| └─ style.css
|
└─ Dockerfile
└─ package.json
└─ vite.config.js
```

## Компонентная архитектура

### BookCard

Компонент отображения информации о книге в виде карточки.

Функции:

- отображение названия книги;
- отображение автора;
- отображение года издания;
- переход к подробной информации. EntitySelect

Компонент с умным выбором и автодополнением для выбора авторов, жанров и издательств.

## **BookForm**

Компонент формы создания и редактирования книги.

### **Обязательные поля:**

- Название книги
- Авторы (один или несколько)
- Жанры (один или несколько)

### **Опциональные поля:**

- Год издания
- ISBN
- Издательство

### **Особенности:**

- Поля для авторов, жанров и издательства используют EntitySelect компонент
- Возможность быстрого создания новых авторов, жанров и издательств прямо из формы
- Поддержка множественного выбора авторов и жанров
- При создании нового элемента через автодополнение, он сразу становится доступным для выбора

## **BaseModal**

Универсальное модальное окно.

Использует механизм слотов Vue для передачи содержимого.

## PageHeader

Переиспользуемый компонент заголовка страницы.

## 3.6 Навигация и интеграция

### Главное меню (App.vue)

В шапке приложения находятся основные ссылки навигации:

- **Книги** → /books
- **Авторы** → /authors
- **Жанры** → /genres
- **Издательства** → /publishers

Отсюда пользователь может быстро перейти к любому разделу приложения.

## Рабочие процессы

### Создание книги с новыми элементами справочников

1. На странице /books нажать **" + Новая книга "**
2. В форме добавления:
  - Ввести название (обязательно)
  - В поле "Авторы" начать вводить имя автора
  - Если автор не найден → нажать **" + Новый "** для создания
  - Новый автор сразу появится в списке и будет выбран

- Повторить для жанров и издательства

### 3. Нажать "Создать книгу"

## Использование слотов

В проекте используются слоты Vue для повышения гибкости компонентов.

Пример:

```
<BaseModal v-model="isOpen" title="Заголовок">
```

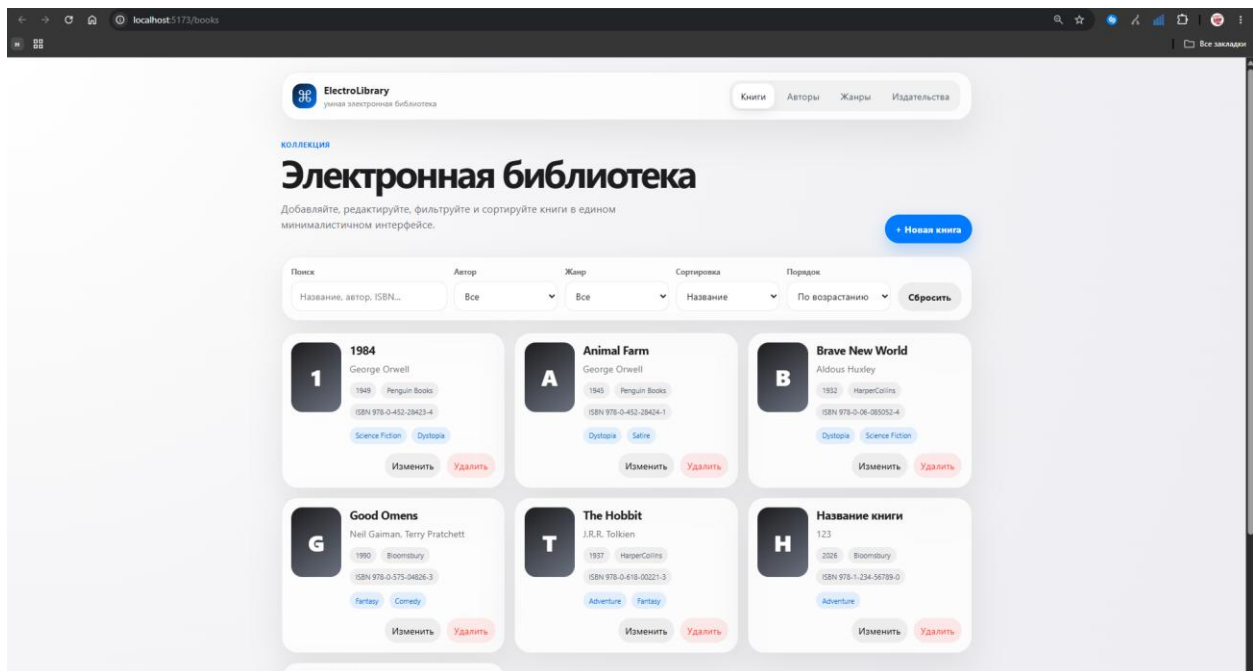
```
<template #default>
```

```
<BookForm />
```

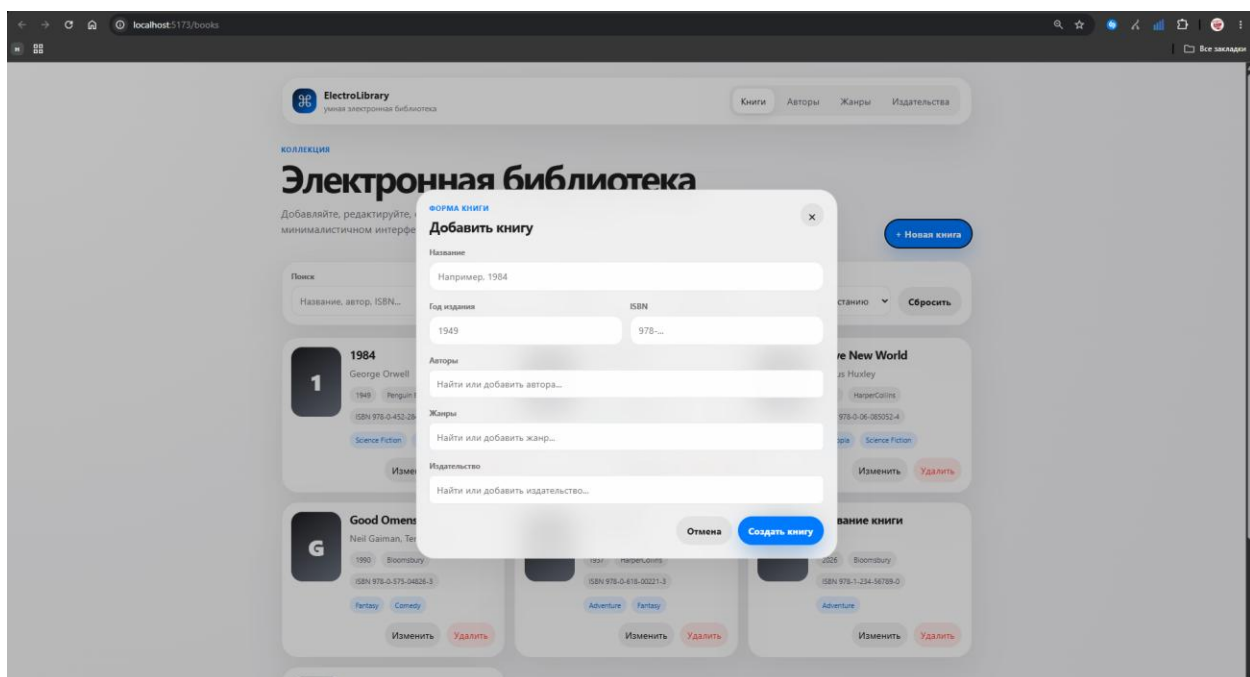
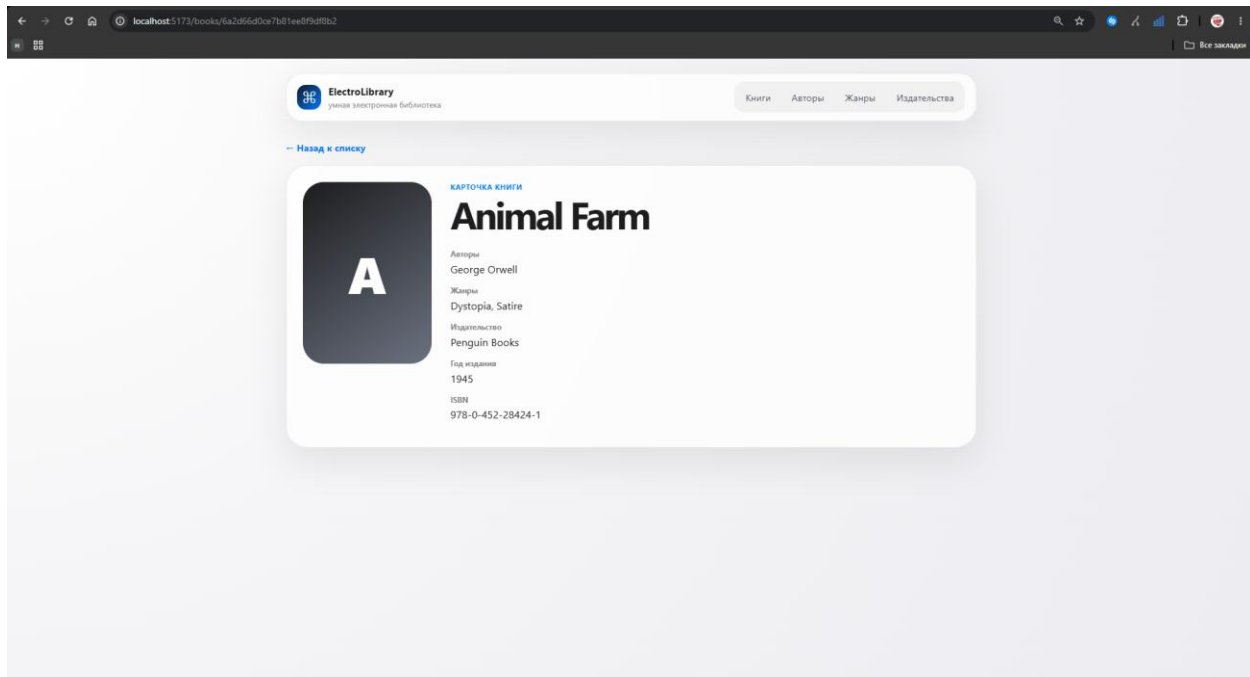
```
</template>
```

```
</BaseModal>
```

Использование слотов позволяет переиспользовать модальные окна и другие компоненты для различных задач.







ElectroLibrary  
электронная библиотека

КнигиАвторыЖанрыИздательства

СПРАВОЧНИКИ

Авторы

Управление авторами библиотеки: добавляйте, редактируйте и удаляйте авторов.

Последний автор

Поиск

Имя, никнейм, email...

Сортировка

Имя

Порядок

По возрастанию

Сбросить

| Имя             | Никнейм     | Email              | Социальные сети        | Действия                               |
|-----------------|-------------|--------------------|------------------------|----------------------------------------|
| 123             | —           | —                  | —                      | <div>Изменить</div> <div>Удалить</div> |
| Aldous Huxley   | —           | huxley@mail.com    | <a href="#">Ссылка</a> | <div>Изменить</div> <div>Удалить</div> |
| George Orwell   | —           | orwell@mail.com    | <a href="#">Ссылка</a> | <div>Изменить</div> <div>Удалить</div> |
| J.R.R. Tolkien  | Tolkien     | tolkien@mail.com   | <a href="#">Ссылка</a> | <div>Изменить</div> <div>Удалить</div> |
| Neil Gaiman     | The Sandman | gaiman@mail.com    | <a href="#">Ссылка</a> | <div>Изменить</div> <div>Удалить</div> |
| Terry Pratchett | —           | pratchett@mail.com | <a href="#">Ссылка</a> | <div>Изменить</div> <div>Удалить</div> |
| Стивен Кинг     | —           | —                  | —                      | <div>Изменить</div> <div>Удалить</div> |

ElectroLibrary  
электронная библиотека

КнигиАвторыЖанрыИздательства

СПРАВОЧНИКИ

Жанры

Управление жанрами библиотеки: добавляйте, редактируйте и удаляйте жанры.

Последний жанр

Поиск

Название жанра...

Сортировка

Название

Порядок

По возрастанию

Сбросить

| Название        | Действия                               |
|-----------------|----------------------------------------|
| Adventure       | <div>Изменить</div> <div>Удалить</div> |
| Comedy          | <div>Изменить</div> <div>Удалить</div> |
| Dystopia        | <div>Изменить</div> <div>Удалить</div> |
| Fantasy         | <div>Изменить</div> <div>Удалить</div> |
| Satire          | <div>Изменить</div> <div>Удалить</div> |
| Science Fiction | <div>Изменить</div> <div>Удалить</div> |
| Horror          | <div>Изменить</div> <div>Удалить</div> |

ElectroLibrary  
умная электронная библиотека

КнигиАвторыЖанрыИздательства

СПРАВОЧНИКИ

Издательства

Управление издательствами библиотеки: добавляйте, редактируйте и удаляйте издательства.

Издательства

Поиск

Сортировка

Порядок

Название, город, email...

Название

По возрастанию

Сбросить

| Название      | Город    | Email                  | Веб-сайт               | Действия |         |
|---------------|----------|------------------------|------------------------|----------|---------|
| Bloomsbury    | London   | support@bloomsbury.com | <a href="#">Ссылка</a> | Изменить | Удалить |
| HarperCollins | New York | info@harpercollins.com | <a href="#">Ссылка</a> | Изменить | Удалить |
| Penguin Books | London   | contact@penguin.co.uk  | <a href="#">Ссылка</a> | Изменить | Удалить |
| Москва        | —        | —                      | —                      | Изменить | Удалить |

— НазадСтраница 1Первая —